

Conditions, dates and tables

Data Science Flipped Classroom - Module 5

Adriana Clavijo Daza

2026-05-21

Logical vectors and comparisons

Logical values

R has a third atomic type alongside numbers and text: **logical**.

Two possible values, **TRUE** and **FALSE** (plus **NA** for unknown).

```
1 c(TRUE, FALSE, TRUE)
```

```
[1] TRUE FALSE TRUE
```

T and **F** work but are not reserved words, so they can be overwritten. Always write the full **TRUE** and **FALSE**.

Why logical vectors matter

Most data questions reduce to:

- Which rows match a condition?
- Which values are missing?
- Where is the price above 1000?

The answer is a logical vector you can pass to `filter()`, `if_else()`, or `case_when()`.

Comparison operators

Operator	Meaning
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	equal to
!=	not equal to

A single = is for assignment in function calls. Equality is ==.

Comparisons are vectorized

The comparison runs element by element:

```
1 x <- c(2, 5, 9, 12)
2 x > 5
```

```
[1] FALSE FALSE TRUE TRUE
```

Recycling kicks in if one side is shorter:

```
1 y <- c(1, 6, 8, 10)
2 x > y
```

```
[1] TRUE FALSE TRUE TRUE
```

Equality with decimals

`==` on doubles can surprise you:

```
1 0.1 + 0.2 == 0.3
```

```
[1] FALSE
```

Use `near()` for numerical equality:

```
1 near(0.1 + 0.2, 0.3)
```

```
[1] TRUE
```

Missing values and `is.na()`

`NA` is contagious. Any comparison with `NA` returns `NA`, not `TRUE` or `FALSE`:

```
1 NA == NA
```

```
[1] NA
```

```
1 NA > 5
```

```
[1] NA
```

To test for missingness, use `is.na()`:

```
1 x <- c(1, NA, 3, NA)
2 is.na(x)
```

```
[1] FALSE  TRUE FALSE  TRUE
```


Membership with %in%

`%in%` tests whether each value on the left appears anywhere on the right.

```
1 species <- c("Droid", "Human", "Wookiee", "Droid")
2 species %in% c("Human", "Wookiee")
```

```
[1] FALSE  TRUE  TRUE  FALSE
```

It is the readable alternative to chaining `==` with `|`:

```
1 species == "Human" | species == "Wookiee"
```

Combining conditions

Operator	Meaning
&	AND, element by element
	OR, element by element
!	NOT

```
1 x <- c(2, 5, 9, 12)
2 x > 3 & x < 10
```

```
[1] FALSE TRUE TRUE FALSE
```

A filtering example

```
1 starwars |>
2   filter(species == "Human", height > 180) |>
3   select(name, height, mass)
```

A tibble: 15 × 3

	name <chr>	height <int>	mass <dbl>
1	Darth Vader	202	136
2	Biggs Darklighter	183	84
3	Obi-Wan Kenobi	182	77
4	Anakin Skywalker	188	84
5	Boba Fett	183	78.2
6	Qui-Gon Jinn	193	89
7	Padmé Amidala	185	45
8	Ric Olié	183	NA
9	Quarsh Panaka	183	NA
10	...	...	...

Commas in `filter()` mean AND. Use `|` explicitly when you want OR.

Conditional values

From a condition to a value

A comparison gives you a vector of `TRUE` and `FALSE`.

Often you want to go one step further and assign each row a different value depending on the answer.

That is what `if_else()` and `case_when()` are for.

if_else()

```
1 if_else(condition, true, false, missing = NULL)
```

- `condition`: a logical vector
- `true`: value returned where the condition is `TRUE`
- `false`: value returned where the condition is `FALSE`
- `missing`: optional value used where the condition is `NA`

if_else() in action

```
1 starwars |>
2   mutate(size = if_else(height > 180, "tall", "short")) |>
3   select(name, height, size) |>
4   head(4)
```

A tibble: 4 × 3

	name	height	size
	<chr>	<int>	<chr>
1	Luke Skywalker	172	short
2	C-3PO	167	short
3	R2-D2	96	short
4	Darth Vader	202	tall

if_else() is type-strict

The `true` and `false` branches must return the same type.

```
1 if_else(c(TRUE, FALSE), "yes", 0)
```

Error in `if\_else()`:

! Can't combine `true` <character> and `false` <double>.

This is a feature, not a bug. Base R's `ifelse()` is more forgiving but can silently change types.

case_when()

When you need more than two outcomes, nested `if_else()` becomes unreadable.

`case_when()` handles multiple conditions in a flat, ordered list:

```
1 case_when(  
2   condition_1 ~ value_1,  
3   condition_2 ~ value_2,  
4   ...  
5   .default = fallback  
6 )
```

Reading case_when()

For each row, R checks conditions **from top to bottom** and returns the value next to the first **TRUE**.

```
1 starwars |>
2   mutate(
3     size = case_when(
4       height < 150 ~ "small",
5       height < 200 ~ "medium",
6       height >= 200 ~ "large"
7     )
8   ) |>
9   select(name, height, size) |>
10  head(4)
```

A tibble: 4 × 3

	name	height	size
	<chr>	<int>	<chr>
1	Luke Skywalker	172	medium
2	C-3PO	167	medium
3	R2-D2	96	small
4	Rey	170	medium

DSFC - Module 5

.default and the catch-all

```
1 x <- c(-1, 5, 12, NA)
2 case_when(
3   x < 0 ~ "negative",
4   x < 10 ~ "small",
5   x >= 10 ~ "large",
6   .default = "unknown"
7 )
```

```
[1] "negative" "small"      "large"      "unknown"
```

Anything not matched by an explicit condition (including `NA`) falls into `.default`. Without it, those values become `NA`.

Order matters

`case_when()` evaluates conditions in order. Put the most specific ones first:

```
1 age <- c(2, 15, 40, 70)
2 case_when(
3   age < 13 ~ "child",
4   age < 20 ~ "teenager",
5   age < 65 ~ "adult",
6   .default = "senior"
7 )
```

```
[1] "child"      "teenager" "adult"     "senior"
```

If `age < 65` came first, everyone under 65 would end up as `adult`.

Dates and times

Why dates are tricky

- Different written formats, 2026-06-04 vs 04/06/2026 vs June 4, 2026
- Time zones and daylight saving time
- Leap years (and leap seconds)
- 30 vs 31 days in a month
- Storing dates as text loses arithmetic

R has dedicated types so you can do real date math.

R's date types

Type	What it stores	Created with
Date	a calendar date	as_date(), ymd()
POSIXct	a date-time, seconds since 1970	as_datetime(), ymd_hms()
POSIXlt	a date-time as a list of components	as.POSIXlt()

In daily use you will mostly meet `Date` and `POSIXct`.

The reference moment

R stores a `Date` as the number of days since 1970-01-01.

```
1 d <- as_date("2026-06-04")
2 d
```

```
[1] "2026-06-04"
```

```
1 unclass(d)
```

```
[1] 20608
```

A `POSIXct` is stored as seconds since the same reference moment, in UTC.

The lubridate package

`lubridate` is the tidyverse package for working with dates and times.

It is loaded with the tidyverse:

```
1 library(tidyverse)
```

Two main families: **parsers** that turn strings into dates, and **accessors** that pull pieces back out.

Parsing dates from strings

The function name tells you the order of year, month, day:

```
1 ymd("2026-06-04")
```

```
[1] "2026-06-04"
```

```
1 dmy("04/06/2026")
```

```
[1] "2026-06-04"
```

```
1 mdy("June 4, 2026")
```

```
[1] "2026-06-04"
```

Add `_hms`, `_hm`, or `_h` for date-times:

```
1 ymd_hms("2026-06-04 14:30:00")
```

```
[1] "2026-06-04 14:30:00 UTC"
```

today() and now()

```
1 today()
```

```
[1] "2026-05-20"
```

```
1 now()
```

```
[1] "2026-05-20 15:27:05 CEST"
```

`today()` returns a `Date`.

`now()` returns a `POSIXct` with your system's time zone.

Extracting components

Function	Returns
<code>year()</code>	the year
<code>month()</code>	the month (number, or label)
<code>day()</code>	the day of the month
<code>hour()</code>	the hour
<code>minute()</code>	the minute
<code>second()</code>	the second

```
1 d <- ymd("2026-06-04")
2 year(d)
```

```
[1] 2026
```

```
1 month(d)
```

```
[1] 6
```

```
1 day(d)
```

```
[1] 4
```

month() with labels

```
1 d <- ymd("2026-06-04")  
2 month(d, label = TRUE)
```

[1] Jun

12 Levels: Jan < Feb < Mar < Apr < May < Jun < Jul < Aug < Sep < ...
< Dec

```
1 month(d, label = TRUE, abbr = FALSE)
```

[1] June

12 Levels: January < February < March < April < May < June < ... <
December

Day of week, day of year

```
1 d <- ymd("2026-06-04")
2
3 wday(d)
```

```
[1] 5
```

```
1 wday(d, label = TRUE)
```

```
[1] Thu
```

```
Levels: Sun < Mon < Tue < Wed < Thu < Fri < Sat
```

```
1 yday(d)
```

```
[1] 155
```

`wday()` numbers weeks from Sunday by default. Use `week_start = 1` to start on Monday.

Converting between types

```
1 as_date("2026-06-04 14:30:00")
```

```
[1] "2026-06-04"
```

```
1 as_datetime("2026-06-04")
```

```
[1] "2026-06-04 UTC"
```

`as_date()` drops the time.

`as_datetime()` lifts a date into a date-time at midnight UTC.

Time zones with `with_tz()`

`with_tz()` shows the **same instant** in another time zone:

```
1 t <- ymd_hms("2026-06-04 14:30:00", tz = "Europe/Zurich")
2 t
```

```
[1] "2026-06-04 14:30:00 CEST"
```

```
1 with_tz(t, "America/New_York")
```

```
[1] "2026-06-04 08:30:00 EDT"
```

Use `force_tz()` if you want to keep the clock reading the same and change the zone label, for example to fix data that was tagged with the wrong zone.

Date arithmetic

Subtracting two dates gives a duration:

```
1 ymd("2026-07-15") - ymd("2026-06-04")
```

Time difference of 41 days

Add days, weeks, or months with `lubridate` helpers:

```
1 ymd("2026-06-04") + days(10)
```

```
[1] "2026-06-14"
```

```
1 ymd("2026-06-04") + weeks(2)
```

```
[1] "2026-06-18"
```

```
1 ymd("2026-06-04") + months(1)
```

```
[1] "2026-07-04"
```

Displaying tables

Plain printing is not enough

By default, R prints a data frame as plain text:

```
1 starwars |>  
2   select(name, height, mass, species) |>  
3   head(3)
```

```
# A tibble: 3 × 4  
  name          height  mass species  
  <chr>         <int> <dbl> <chr>  
1 Luke Skywalker    172     77 Human  
2 C-3PO             167     75 Droid  
3 R2-D2             96     32 Droid
```

Fine while exploring, not what you want in a report or paper.

Two table packages

`knitr::kable()`

- Already there with every Quarto setup
- Quick, minimal, works in PDF and HTML
- Few formatting options

`gt::gt()`

- A grammar of tables, similar to ggplot
- Rich formatting: spanners, footnotes, styles
- Best output in HTML

For everyday reports, reach for `kable()`. For publication-quality tables in HTML, reach for `gt`.

knitr::kable()

```
1 starwars |>
2   select(name, height, mass, species) |>
3   head(4) |>
4   knitr::kable()
```

name	height	mass	species
Luke Skywalker	172	77	Human
C-3PO	167	75	Droid
R2-D2	96	32	Droid
Darth Vader	202	136	Human

Key `kable()` arguments

Argument	What it controls
<code>x</code>	the data
<code>format</code>	"html", "latex", "pipe", "markdown"
<code>digits</code>	number of decimal places
<code>col.names</code>	custom column labels
<code>align</code>	column alignment: "l", "c", "r"
<code>caption</code>	table caption

kable() with formatting

```
1 starwars |>
2   select(name, height, mass, species) |>
3   head(4) |>
4   knitr::kable(
5     col.names = c("Name", "Height (cm)", "Mass (kg)", "Species"),
6     align      = c("l", "r", "r", "l"),
7     digits     = 1
8   )
```

Name	Height (cm)	Mass (kg)	Species
Luke Skywalker	172	77	Human
C-3PO	167	75	Droid
R2-D2	96	32	Droid
Darth Vader	202	136	Human

gt::gt()

```
1 starwars |>
2   select(name, height, mass, species) |>
3   head(4) |>
4   gt()
```

name	height	mass	species
Luke Skywalker	172	77	Human
C-3PO	167	75	Droid
R2-D2	96	32	Droid
Darth Vader	202	136	Human

The anatomy of a gt table

A `gt` table has named parts you can address one at a time:

- table header: title and subtitle
- column labels: the column headers
- stub: a row-label column on the left
- body: the data cells
- footer: footnotes and source notes

A more polished gt table

```
1 starwars |>
2   select(name, height, mass, species) |>
3   head(4) |>
4   gt() |>
5   tab_header(
6     title      = "Star Wars characters",
7     subtitle   = "First four rows"
8   ) |>
9   cols_label(
10    name       = "Name",
11    height     = "Height (cm)",
12    mass       = "Mass (kg)",
13    species    = "Species"
14  ) |>
15  fmt_number(columns = c(height, mass), decimals = 1)
```

Star Wars characters			
First four rows			
Name	Height (cm)	Mass (kg)	Species
Luke Skywalker	172.0	77.0	Human

DSFC - Module 5

Star Wars characters

First four rows

Name	Height (cm)	Mass (kg)	Species
C-3PO	167.0	75.0	Droid
R2-D2	96.0	32.0	Droid
Darth Vader	202.0	136.0	Human

Useful gt helpers

Function	Purpose
<code>tab_header()</code>	title and subtitle
<code>cols_label()</code>	rename columns for display
<code>fmt_number()</code>	format numbers
<code>fmt_percent()</code>	format percentages
<code>fmt_currency()</code>	format currency
<code>tab_spanner()</code>	grouping label across columns
<code>tab_footnote()</code>	footnote attached to a cell
<code>tab_source_note()</code>	source line at the bottom
<code>gtsave()</code>	save as HTML, PNG, or PDF

Captions, labels, cross-references

Why label a table?

In a long document, you often write something like:

“See Table 2 for the breakdown by species.”

Quarto can **number tables automatically** and turn `@tbl-species` into “Table 2”, with a live link. You just need to label them properly.

Chunk options for a table

In any code chunk that produces a table, set two options:

```
1 ```{r}
2 #| label: tbl-species
3 #| tbl-cap: "Average height by species"
4
5 starwars |>
6   group_by(species) |>
7   summarise(height = mean(height, na.rm = TRUE)) |>
8   knitr::kable()
9 ```
```

- `label` must start with `tbl-` for Quarto to treat it as a table
- `tbl-cap` is the caption text

Cross-referencing in text

In the document body, refer to the table with `@tbl-species`:

- 1 Average height differs noticeably between species
- 2 (see @tbl-species).

Quarto renders this as:

Average height differs noticeably between species (see Table 1).

Captions for gt tables

`gt` recognises the chunk's `tbl-cap` option, so the same pattern works:

```
1 ```{r}
2 #| label: tbl-starwars-gt
3 #| tbl-cap: "Star Wars characters"
4
5 starwars |>
6   select(name, height, mass, species) |>
7   head(4) |>
8   gt()
9 ```
```

Use the chunk options for the caption, not `tab_header()`, when you want Quarto to handle the numbering and the cross-reference.

Sub-captions for several tables

You can lay out **several tables side by side** with sub-labels:

```
1  ```{r}
2  #| label: tbl-starwars
3  #| tbl-cap: "Star Wars summaries"
4  #| tbl-subcap:
5  #|   - "By species"
6  #|   - "By homeworld"
7  #| layout-ncol: 2
8
9  starwars |> count(species) |> knitr::kable()
10 starwars |> count(homeworld) |> knitr::kable()
11 ```
```

Cross-reference the pair with `@tbl-starwars`, or a single sub-table with `@tbl-starwars-1`.

The same pattern for figures

Figures use the same mechanism, with a `fig-` prefix instead:

```
1 ```{r}
2 #| label: fig-height-mass
3 #| fig-cap: "Height vs mass of Star Wars characters"
4
5 starwars |>
6   ggplot(aes(height, mass)) +
7   geom_point()
8 ```
```

- `label` must start with `fig-`
- `fig-cap` is the caption text

Cross-referencing figures

Refer to a figure with `@fig-...`:

- 1 Height and mass are positively correlated
- 2 (see @fig-height-mass).

Quarto renders this as:

Height and mass are positively correlated (see Figure 1).

Several figures side by side

Use `fig-subcap` and `layout-ncol`, just like for tables:

```
1  ```{r}
2  #| label: fig-starwars
3  #| fig-cap: "Distributions of body measurements"
4  #| fig-subcap:
5  #|   - "Height"
6  #|   - "Mass"
7  #| layout-ncol: 2
8
9  ggplot(starwars, aes(height)) + geom_histogram()
10 ggplot(starwars, aes(mass))   + geom_histogram()
11 ```
```

Cross-reference the pair with `@fig-starwars`, or a single panel with `@fig-starwars-1`.

A checklist for nice tables and figures

- Pick `kable()` for quick tables, `gt()` for ones that need polish
- Always set `label` (`tbl-` or `fig-`) and a caption (`tbl-cap` or `fig-cap`)
- Cross-reference with `@tbl-...` or `@fig-...`, never by typing “Table 2” or “Figure 1” by hand
- Format numbers so the reader can compare them at a glance
- Tables and figures both renumber themselves when you move them around

Learn more

<https://lubridate.tidyverse.org>

<https://gt.rstudio.com>

<https://quarto.org/docs/authoring/cross-references.html>

Questions?